

Session III -

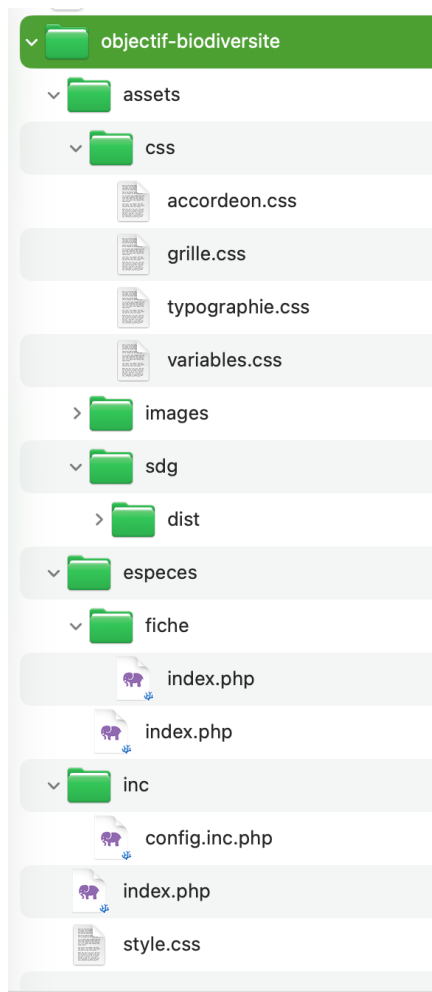
Démarrer le projet *Objectif Biodiversité*

La base de données

Déposée en format unique sur Timunix3, il reste encore un peu de travail à faire pour la compléter aussi, pour l'instant vous n'y avez accès qu'en mode lecture.

```
$strHost      = 'timunix3.cegép-ste-foy.qc.ca';  
$strUser      = "26_rpn11_LFVQ";  
$strPassword  = "biodiversite";  
$strBD        = "26_rpn11_LFVQ";
```

Bâtir la structure de répertoires



Créer le USER d'équipe sur Timunix3

Chaque équipe s'est déterminé un nom d'équipe et un mot de passe.

1. Créer avec le professeur le *user* de l'équipe sur Timunix3, c'est le lieu de mise en commun du code en parallèle avec Github.
2. Configurer FileZilla pour l'accès ftp au dossier d'équipe sur Timunix3.

Créer un répertoire Github pour l'équipe

1. L'équipe décide qui des coéquipiers aura la responsabilité du dépôt Github.
2. Le responsable crée le dépôt **privé** au nom de **p1-objectif-biodiversite** AVEC un README mais SANS fichier .gitignore
3. Le responsable ajoute comme **collaborateurs** :
la ou le coéquipier ainsi que la prof (evefevrier)
4. **Faire l'exercice *Départ avec les branches***
Ceci doit être fait aujourd'hui.
5. Le responsable implémente la structure de répertoires et fait le premier commit **"init P1"** (ou message équivalent).
Ceci doit être fait avant le mercredi 6 octobre.

Tester l'utilisation des branches

Pour se sentir à l'aise dans le travail d'équipe, pratiquez les commandes essentielles en réalisant le petit exercice détaillé dans les pages qui suivent.



Réchauffement avec les branches de versionnage

Étape 0 : Initialisation

1. **Le responsable Github (profil A)** a créé un dépôt public sur GitHub et invite sa-son coéquipier.e (*Settings > Collaborators > Add people*).
2. **Le responsable Github** clone le dépôt, remplace le fichier `README.md` par celui fourni pour le projet et ajoute le fichier `.gitignore` puis fait son premier commit et le pousse sur la branche `main`.
3. **Coéquipier.e (profil B)** clone le dépôt à son tour.

Étape 1 : Le travail en parallèle sur des fichiers distincts

Chaque étudiant va créer sa propre branche pour ajouter sa fiche profil.

- **Responsable Github - A:**
 1. Crée et bascule sur sa branche : `git switch -c feature-profil-A`
 2. Crée un **nouveau fichier** `profil_A.txt` et écrit son nom dedans.
 3. Sauvegarde et valide : `git add profil_A.txt` puis `git commit -m "Ajout du profil de A"`
 4. Envoie sur GitHub : `git push origin feature-profil-A`
- **Coéquipier.e - B:**
 1. Crée et bascule sur sa branche : `git switch -c feature-profil-B`
 2. Crée un **nouveau fichier** `profil_B.txt` et écrit son nom dedans.
 3. Sauvegarde et valide : `git add profil_B.txt` puis `git commit -m "Ajout du profil de B"`
 4. Envoie sur GitHub : `git push origin feature-profil-B`

Nota bene : l'utilisation de `git checkout -b` (Obsolète) :

Bien que très populaire, la commande `checkout` fait trop de choses à la fois (gérer les branches et restaurer des fichiers), ce qui perturbe les étudiants. Depuis quelques années, Git a introduit la commande moderne `git switch -c`, exclusivement dédiée à la création et au changement de branche.

Étape 2 : Pull Requests et Revue de code (15 minutes)

1. Sur GitHub, chacun ouvre une **Pull Request** vers la branche `main`.

2. **Règle absolue de l'équipe :**

On ne fusionne pas son propre PR (Pull Request).

Responsable Github (A) va sur la PR de **coéquipier.e (B)**, vérifie le code, laisse un commentaire sympa et clique sur **Approve**.

Coéquipier.e (B) fait de même pour A.

3. Une fois approuvées, ils cliquent sur **Merge pull request**. [7]

Étape 3 : La synchronisation locale (5 minutes)

Pour finir proprement, synchroniser en local avec GitHub :

```
git switch main  
git pull origin main
```

Vous verrez alors apparaître les fichiers des deux profils en local.

En cas de difficulté, bogue ou problème inattendu, lever la main!